从门电路到大模型

FPGA 部署 AI 模型的学习路线

为有 AI 基础、没有硬件基础的你设计：先把数字电路、FPGA 工具和加速器设计的基础打牢，板子已经到了，基础阶段就穿插上板实验；之后依次部署分类、检测与分割、多模态和语言模型。 量化不是这条路线的重点：先用最常见的 INT8 量化自己跑通，重心放在把量化后的模型变成在 FPGA 上运行的电路。



起点

有 AI 基础，熟悉 Python 和 PyTorch；本科学过部分数电、模电，没有 HDL 和 FPGA 经验。手上有两块板：主力是实验室里你独占的 EdgeBoard FZ5D，自己的 ALINX AXU4EVB-E 用来对照，体会不同硬件环境的差别。

终点

能把量化稀疏后的分类、检测、多模态和语言模型部署到 FPGA，并能自己设计加速器。

分工

量化同学的进度不确定，路线不等他：前期用公开工具自己做最常见的 INT8 量化，按一份默认格式约定推进。他的成果到了再替换、对比。

工作量

S0	S1	S2	S3	S4	S5	S6	S7	S8	S9
20h	70h	66h	65h	60h	46h	150h	160h	160h	200h

整条主线预计约 **997 小时**。按已勾选的任务估算，已完成约 **32 小时**，剩余约 965 小时，当前在 **S1 数电与 Verilog**。板子（ALINX AXU4EVB-E V3.0）已在 S1 期间到货，S1 到 S3 里已经插入上板实验，不必等到 S5。主力板是你独占的 EdgeBoard FZ5D，AXU4EVB-E 每个阶段做一次对照，体会换硬件环境的差别（见两块板怎么配合）。可选的 CUDA 支线另需约 40 小时，不计入主线。
按和 AI 助手的分工把重复性工作交出去，主线预计还能省约 **156 小时**，主要省在 S5 以后的工程胶水和实验脚本上。终点指标：S9 在主力板上 0.5B INT4 模型解码预期 **40–60 tok/s**（见 S9 预期表）。

小时数是按零基础估算的，误差可能有三到五成，做完 S1 后可以按实际用时校准。 时间零碎时这样安排：只有 1 小时，就做标着“1 小时可做”的任务，比如刷题、读论文、做统计； 有半天的时候，留给装工具、跑综合、上板调试这类一旦中断就要重新进入状态的任务。 每次结束前在阶段笔记里写一句“下次从哪继续”，下次能省下十几分钟的回想时间。 资料上，S1、S2 优先用标着“中文”的资料降低门槛，S3 以后逐步过渡到英文原版和论文。

阶段 0

全景认知与路线选型

已完成　约 20 小时　全部可在仿真中完成　概览

搞清楚“把模型放到 FPGA 上”有哪几条技术路线，各自适合哪个阶段。

阶段 1

数字电路回炉 + Verilog

当前阶段　约 70 小时　以仿真为主，其中 1 项任务上板　硬件

能用 Verilog 写出组合电路、时序电路和状态机，跑仿真并看懂波形。

后面即使主要写 HLS，调试、读综合报告、写接口胶水逻辑也离不开 RTL 思维。这一阶段要建立的直觉是：硬件是同时工作的电路，不是逐行执行的程序。

要掌握的概念

- **组合逻辑与时序逻辑**：触发器、寄存器、时钟、复位。
- **阻塞赋值与非阻塞赋值**：写错是新手最常见的仿真/综合不一致来源。
- **有限状态机**：控制逻辑的标准写法。
- **时序**：建立/保持时间、关键路径、最高频率，以及用流水线提高频率。
- **定点数**：补码、Q 格式、乘加器（MAC）与累加位宽。
- **握手协议**：valid/ready，后面 AXI-Stream 的基础。

动手任务

- [x] ~~刷 HDLBits 到 Sequential Logic 章节结束（约 120 题）~~ 1 小时可做 仿真

  网页上做题、自动判对错，不用装任何软件，用来熟悉 Verilog 语法和组合、时序电路的基本写法。

  1. 在 hdlbits.01xz.net 注册，按左侧目录从 Getting Started 一直做到 Circuits → Sequential Logic 结束：锁存器与触发器、计数器、移位寄存器、状态机。
  2. 每题先自己写，提交后看 Simulation 里的波形对比，不对就找第一处 mismatch 的时刻。
  3. 卡住超过 20 分钟再看讨论区或问 AI。问它“我哪里错了”，不要直接要答案。
  4. 把踩过的坑记进本阶段笔记：阻塞和非阻塞赋值、always @(*) 漏写分支生成锁存器、状态机的写法。

  做完的标志 Sequential Logic 下的题目基本全部打勾，状态机的几道大题可以先留着。

- [ ] 装好仿真工具（Icarus Verilog + GTKWave），跑通第一个 testbench：读懂配套目录 s1/01_first_tb 里现成的计数器测试，再自己加一项检查 1 小时可做 仿真

  硬件出了错很难在板上查原因，每个电路都要先在电脑上仿真。testbench 就是硬件的测试代码，相当于 pytest：产生时钟、给输入、和参考答案比对、导出波形。这个任务没有现成的 testbench 要你去找：配套目录里给了一个完整的示例，你读懂它，再自己加一项检查。

  1. 安装：sudo apt install iverilog gtkwave，运行 iverilog -V 能看到版本号。
  2. 进入配套目录运行 make，终端打印 PASS。
  3. 运行 make wave，在 GTKWave 里把 clk、rst、q 加进波形窗口，找到复位结束、15 回到 0、使能关闭这三个时刻。
  4. 故意把计数器的 +1 改成 +2，看 testbench 怎么报错，再改回来。
  5. 自己加一项检查：计数到一半时复位一拍，确认 q 回到 0。

  做完的标志 make 打印 PASS；能说清 testbench 里六段代码各做什么；自己加的复位检查能通过。

  配套目录：`s1/01_first_tb`（仓库 main 分支，`git clone https://github.com/yanhexiong/FPGA_ROUTE.git` 即可拿到），详细说明见目录里的 README

- [ ] 写一个带 valid/ready 握手的同步 FIFO 并仿真 1 小时可做 仿真

  加速器里到处是 FIFO；valid/ready 握手是 AXI-Stream 的核心规则，S2、S3、S5 都会反复用到。

  1. 读配套目录的 README，弄清规格：满时 s_ready 为 0，非空时 m_data 始终是队头。
  2. 在纸上画结构：存储阵列、读写指针，想清楚空和满怎么区分。
  3. 只写 sync_fifo.v，接口已经定好；tb_sync_fifo.v 是验收测试，不用改。
  4. 运行 make，没通过就看第一条 FAIL 的时间点，打开波形对照。
  5. 通过后想一想：满时也允许边读边写，s_ready 要怎么改，会带来什么时序问题。

  做完的标志 make 打印 PASS（写满、读空、3000 拍随机读写都和参考模型一致），能画出自己的 FIFO 结构图。

  配套目录：`s1/02_fifo`（仓库 main 分支，`git clone https://github.com/yanhexiong/FPGA_ROUTE.git` 即可拿到），详细说明见目录里的 README

- [ ] 写 INT8 乘加单元（32 位累加），用 Python 生成随机数据比对结果 1 小时可做 仿真

  神经网络算到底层都是 acc += a*b。这个任务还会用到后面一直沿用的验证方法：Python 算参考答案，硬件仿真逐拍比对。

  1. 读 gen_vectors.py，弄清参考模型怎么算、每行十六进制里各段是什么。
  2. 写 mac.v：重点是符号，乘积和中间变量都要声明成 signed，否则负数会被当成大正数。
  3. 运行 make：它先用 Python 生成 2000 拍测试向量，再跑仿真逐拍比对。
  4. 通过后去掉乘积的 signed 再跑一次，亲眼看看符号错误长什么样。

  做完的标志 make 打印 PASS；能解释 8 位乘 8 位为什么用 16 位存积，累加器为什么用 32 位。

  配套目录：`s1/03_mac`（仓库 main 分支，`git clone https://github.com/yanhexiong/FPGA_ROUTE.git` 即可拿到），详细说明见目录里的 README

- [ ] 安装 Vivado（版本见下方“待讨论”），做第一个上板实验：PL 侧 LED 流水灯加按键控制，走通 综合 → 比特流 → JTAG 下载。先在 AXU4EVB-E 上照 ALINX 教程跑通，再移植到主力板 FZ5D：从米尔原理图查哪些 LED、按键接在 PL 上，只改约束文件。这是第一次体会“同一份 RTL 换板子要改什么” 需板

  走通一次 FPGA 的完整流程：RTL → 综合 → 实现 → 比特流 → JTAG 下载。再把同一份 RTL 移植到第二块板，体会换板要改什么。

  1. 在 Ubuntu 22.04 虚拟机里装 Vivado 2023.1：选 Vivado ML Standard，器件只勾 Zynq UltraScale+ MPSoC，可以省下大量磁盘。
  2. 照 ALINX 的 LED 教程，记下 AXU4EVB-E 的芯片型号、PL 时钟、LED 和按键的引脚。
  3. 自己写流水灯 RTL：分频到约 4 Hz，按键先经过两级寄存器同步，再控制暂停。
  4. 写约束文件，综合、实现、生成比特流，在 Hardware Manager 里下载，LED 开始流动。
  5. 查米尔原理图，RTL 不改，只重写约束文件，在 FZ5D 上再跑一遍；填配套目录里的差异表。

  做完的标志 两块板上的流水灯都能运行，按键能控制；有一张填好的两板差异表。

  配套目录：`s1/05_led_board`（仓库 main 分支，`git clone https://github.com/yanhexiong/FPGA_ROUTE.git` 即可拿到），详细说明见目录里的 README

- [ ] 进阶：写 4×4 脉动阵列做矩阵乘，用 cocotb（Python 写 testbench）验证 仿真

  脉动阵列是 TPU 和大多数 AI 加速器的计算核心。testbench 换成 cocotb，用 Python 写测试，参考答案直接用 NumPy 算。

  1. pip install cocotb numpy（建议用虚拟环境），另外要有 make。
  2. 在纸上手推 3 拍：A 取单位矩阵，看 PE(0,0)、PE(0,1)、PE(1,1) 各在哪一拍拿到哪个数。
  3. 写 pe.v（MAC 加两个传递寄存器）和 systolic4x4.v（用 generate 例化 16 个 PE 并连线）。
  4. 运行 make：先过单位矩阵测试，再过 20 组随机矩阵和连续两次矩阵乘。
  5. 读懂 test_systolic.py 里输入是怎么错开喂进阵列的（skew）。

  做完的标志 cocotb 显示 TESTS=3 PASS=3；能画出数据在阵列里怎么流动，说清第几拍哪个 PE 在算哪一项。

  配套目录：`s1/04_systolic_cocotb`（仓库 main 分支，`git clone https://github.com/yanhexiong/FPGA_ROUTE.git` 即可拿到），详细说明见目录里的 README

验收标准

4×4 脉动阵列 GEMM 的 RTL 和自动比对的 testbench，能对着波形讲清楚数据怎么流动。

和 AI 助手的分工 预计省约 5 小时

自己动手

- FIFO、MAC、脉动阵列的 RTL 从零自己写
- 看波形定位 bug，自己找到原因

交给 AI，你来验收

- 装 Icarus / Verilator / cocotb 的环境脚本和 Makefile
- Python 随机激励和自动比对脚本

- 你写完 RTL 后让它 review：锁存器、阻塞赋值、复位遗漏

别做或少做

- 指定范围以外继续刷 HDLBits
- 手动截图波形贴进报告

学习资料

- HDLBits https://hdlbits.01xz.net
  在线刷题，即时判题，最适合零基础
- HDLBits 中文版电子书 https://github.com/OpenIC-SIG/HDLBits-Chinese-gitbook 中文
  HDLBits 题目的中文翻译，和原站对照着刷
- Harris & Harris《数字设计和计算机体系结构》第 1–5 章 中文
  补数电基础，用中文版即可
- 一生一芯（只做前期内容） https://ysyx.oscc.cc 中文
  做前期的 Linux、C、数字电路和 Verilator 部分；后面的处理器设计和流片与本路线关系不大
- cocotb https://www.cocotb.org
  用 Python 写 testbench，适合有 AI 背景的人
- Nandland https://nandland.com
  FPGA 入门讲解清楚

容易踩的坑

- 把 Verilog 当 C 写：for 循环会被展开成一堆硬件，而不是循环执行。
- 只看仿真打印的结果，不看波形。
- 忽略复位和时钟，仿真能过、上板出错。

待讨论

- 工具版本（建议）：Vivado、Vitis、PetaLinux 统一用 2023.1。这是 Vitis AI 3.5 + DPU 4.1 官方验证过的组合，两块板的 XCZU5EV、XCZU4EV 都在免费档位内，不用买 license。ALINX 同系列的出厂例程多为 2020.1，旧工程可以在 2023.1 里升级 IP，或者用板卡约束和 PS 预设重建。系统用 Ubuntu 22.04，24.04 不在 2023.1 的支持列表里。2023.1 是项目内冻结的基线，中途不升级；S6 的 DPU 基线做完后，计划有准备地升级一次再做 S7–S9。

阶段 2

FPGA 架构、Vivado 流程与 AXI 总线

约 66 小时　以仿真为主，其中 3 项任务上板　硬件

理解 FPGA 内部资源，能把设计从 RTL 跑到比特流，看懂资源和时序报告，掌握 AXI 总线。

资源报告会告诉你这块板子到底能放多大的加速器。板子已经到了，这一阶段的系统实验直接上板，用 PS 端的裸机程序验证。

要掌握的概念

- **LUT、FF、CLB**：通用逻辑资源。
- **DSP48**：乘加的主力，数量基本决定了算力上限。
- **BRAM / URAM**：片上存储，决定能缓存多少权重和激活。
- **布线与时序收敛**：WNS 为负说明时序没满足。
- **Zynq 架构**：PS（ARM 处理器）+ PL（可编程逻辑），AI 部署最常用的形态。
- AXI4 搬数据、AXI4-Lite 配寄存器、AXI4-Stream 传数据流；DMA；DDR 带宽。

动手任务

- [ ] 给两块板各建一个可以反复复用的 Vivado 工程模板：FZ5D 从米尔资料包里的 Vivado 工程或 HDF 导入 PS 配置，AXU4EVB-E 用 ALINX 的。用 2023.1 重建、导出 XSA。同一份 RTL 只换约束和 PS 配置　仿真

  之后每个实验都从模板起步，省掉每次重新配置 PS。两块板各一份，这本身就是第一次系统地对比两套板级配置。

  1. AXU4EVB-E：照 ALINX 教程新建工程，在 Block Design 里放 Zynq UltraScale+ MPSoC IP，应用 ALINX 给的 PS 预设，核对 DDR、时钟、串口和 SD 卡的配置。
  2. FZ5D：在米尔资料包里找 Vivado 工程或 HDF。有工程就用 2023.1 打开，Report IP Status 后把 IP 全部升级；只有 HDF 的话，它其实是个压缩包，里面的 psu_init 文件记录了 PS 配置，可以对照着在新工程里重新配。
  3. 对着原理图核对 FZ5D 的 DDR 型号、位宽和速率。这一步自己做，不要交给 AI。
  4. 两个工程都用 File → Project → Write Tcl 导出成脚本，以后一条命令就能重建（整理脚本可以交给 AI）。
  5. 生成比特流后 File → Export → Export Hardware，勾选包含比特流，得到 XSA。

  做完的标志　两份 Tcl 脚本都能在空目录里重建工程；两个 XSA；笔记里记下每块板 PS 配置的要点。

- [ ] 把 S1 的脉动阵列分别综合实现到 XCZU5EV 和 XCZU4EV，读资源利用率和时序报告，看同一个设计在两颗芯片上的差别　仿真

  资源报告和时序报告告诉你一块芯片到底能放多大的加速器。同一个设计放到两颗芯片上，差别一目了然。

  1. 新建一个纯 RTL 工程，加入 S1 的 pe.v 和 systolic4x4.v。
  2. 阵列端口太多，超过芯片引脚数，所以用 out-of-context 模式综合：在综合设置里加 -mode out_of_context。
  3. 加一条时钟约束，比如 create_clock -period 3.333 -name clk [get_ports clk]，即目标 300 MHz。
  4. 跑综合和实现，看 Report Utilization（LUT、FF、DSP、BRAM）和 Report Timing Summary（WNS）。
  5. 把目标器件换成另一颗芯片，再跑一遍。

  做完的标志　一张表：两颗芯片 × LUT、FF、DSP、WNS、可达频率（约等于 1000 ÷（时钟周期 − WNS）MHz）。

- [ ] 把阵列逐步加大，记录 DSP 用量和最高频率的变化　1 小时可做　仿真

  看清资源和频率怎么随阵列规模变化，找到每颗芯片上的瓶颈是 DSP、布线还是时序。

  1. 把阵列改成参数化的：S1 写死了 4×4，改成 parameter N。
  2. N 取 4、8、12、16……逐个做 out-of-context 综合加实现，直到 DSP 用满或者时序崩掉。批量跑的 Tcl 脚本和提取数字的脚本交给 AI。
  3. 两颗芯片各画一张图：横轴 N，纵轴 DSP 用量和可达频率。

  做完的标志　两张曲线图，加一段解释：每颗芯片最大能放多大的阵列，卡在哪里。

- [ ] 在 FZ5D 上用 Block Design 搭 Zynq PS + 自定义 AXI-Lite IP 的系统，上板后用 Vitis 裸机 C 程序读写寄存器　需板

  PS 上的 CPU 通过 AXI-Lite 读写寄存器来控制加速器，这是后面所有加速器的控制方式。

  1. Tools → Create and Package New IP → Create AXI4 Peripheral，生成一个带 4 个寄存器的 AXI-Lite 模板。向导最后一页勾选 Verify Peripheral IP using AXI4 VIP，下一个任务会用到。
  2. 在模板里加一点自己的逻辑，比如把寄存器 0 和寄存器 1 的和放进寄存器 2。
  3. 在 FZ5D 的模板工程里新建 Block Design：Zynq MPSoC 加上你的 IP，点 Run Connection Automation 自动连线。
  4. 生成比特流，导出 XSA。在 Address Editor 里记下 IP 的基地址。
  5. 在 Vitis 2023.1 里用 XSA 新建 platform，再建一个 Hello World 应用，用 Xil_Out32、Xil_In32 读写寄存器，结果从串口打印出来。

  做完的标志　串口打印出正确的加法结果。

- [ ] 用 AXI VIP 仿真验证 AXI-Lite 寄存器读写　仿真

  上板前先在仿真里确认 AXI 握手对不对，比在板上猜容易得多。

  1. 用上一个任务向导生成的 VIP 示例工程，或者自己在 testbench 里放一个 AXI VIP（master 模式）接到你的 IP。
  2. 写入几个值再读回来比对。
  3. 在波形里找到 AW、W、B、AR、R 五个通道各自的 valid/ready 握手，对照 S1 的 FIFO 理解。

  做完的标志　仿真里写入的值能正确读回；能指出一次写操作里三个通道的握手顺序。

- [ ] 用 ILA 在板上抓一次 AXI-Lite 读写波形，和仿真波形对照　需板

  ILA 是插在芯片里的逻辑分析仪，板上出问题时用它看真实信号。

  1. 在 Block Design 里右键你的 IP 的 S_AXI 接口 → Debug，再点 Run Connection Automation，自动插入 System ILA。
  2. 重新生成比特流，下载后在 Hardware Manager 里把触发条件设成 AWVALID = 1。
  3. 运行上个任务的 Vitis 程序，ILA 抓到一次写操作。
  4. 和 VIP 仿真的波形对照，看握手顺序是否一致。

  做完的标志　有两张截图：仿真波形和 ILA 抓到的波形，握手顺序一致。

- [ ] 把 AXI-Lite 工程在 AXU4EVB-E 上也跑一遍，列一张两块板 PS 配置的差异表：DDR 型号和位宽、参考时钟、MIO 外设分配、启动方式　1 小时可做　需板

  同一个系统在第二块板上再搭一遍，体会 PS 配置为什么是板级的。

  1. 用 AXU4EVB-E 的模板工程，复用同一个 IP，重建同样的 Block Design。
  2. 生成比特流，在 Vitis 里换成这块板的 platform，运行同一个程序。
  3. 在 S1 的差异表里补充：DDR 型号、位宽和速率，PS 参考时钟，串口用的是哪组 MIO，启动方式和拨码开关，IP 的基地址是否相同。

  做完的标志　两块板串口输出一致，差异表补完。

验收标准

一份报告：你的阵列在 XCZU5EV 和 XCZU4EV 上各能放多大、跑多快，瓶颈是 DSP、BRAM 还是时序；外加两块板的 2023.1 硬件平台（XSA）、一个在两块板上都跑通的 PS 读写自定义 IP 的裸机工程，和一张两块板的 PS 配置差异表。

和 AI 助手的分工　预计省约 10 小时

自己动手

- 读懂资源和时序报告，弄懂 AXI 握手
- 对照 ALINX 和米尔的原理图亲自核对引脚约束、电平标准和 PS 时钟、DDR 配置

交给 AI，你来验收

- 把工程改成 Vivado Tcl 脚本（非工程模式），一条命令重建
- 批量综合不同阵列规模，从报告里自动提取 DSP、频率，生成表格
- 裸机读写寄存器的 C 程序样板

别做或少做

- 在 GUI 里反复手点重建工程
- 手抄报告数字到表格

学习资料

- ALINX 给 AXU4EVB-E 配套的例程和教程 中文
  优先用这一套：引脚约束、PS 配置、例程都对应你的板子。B 站也有 ALINX 的 MPSoC 视频教程
- 米尔 FZ5 资料包（随板光盘或向米尔索取） 中文
  移植到 FZ5D 时要用：原理图、Vivado 工程或 HDF 里的 PS 侧 DDR 和引脚配置。没有它，PS 配置只能从原理图倒推
- AMD UG1037 AXI Reference Guide
  AXI 协议的实用说明
- AMD UG1085 Zynq UltraScale+ 技术参考手册
  查阅用，不必通读
- The Zynq Book
  免费电子书，基于 Zynq-7000，概念通用

容易踩的坑

- 随手装最新版。Vivado、Vitis、PetaLinux、ALINX 的 BSP 和以后的 DPU 版本必须匹配，先定版本再装。
- 时序违例的比特流上板后行为不可预测，别抱侥幸。
- FZ5D 直接拿 2019.1 的米尔工程在 2023.1 里升级 IP 后就用。打开后先核对 PS 的 DDR 型号、位宽和时钟参数和原理图一致，再生成比特流。

待讨论

- 安装环境：KVM 虚拟机加串流的 GNOME 桌面可以用，但虚拟机建议给 16 GB 以上内存、300 GB 左右磁盘。板子插在哪台机器上？JTAG 可以 USB 直通进虚拟机，也可以在接板子的机器上跑 hw_server，虚拟机里的 Vivado 通过网络远程连接。

阶段 3

HLS 高层次综合

约 65 小时　以仿真为主，其中 1 项任务上板　算子　硬件

用 C++ 写加速器，通过 pragma 控制并行度，独立完成 GEMM 和卷积 kernel 的优化。

HLS 是做 AI 加速器效率最高的入口，hls4ml 和 FINN 底层也是 HLS。你有软件背景，这一阶段会比 RTL 上手快。

要掌握的概念

- **流程**：C 仿真（csim）→ 综合 → 联合仿真（cosim）→ 导出 IP。
- **PIPELINE 与 II**：II=1 表示每个周期启动一次新迭代。
- UNROLL 与 ARRAY_PARTITION：展开计算的同时要把存储切开，否则访存会卡住并行度。
- **DATAFLOW**：任务级流水，多个函数同时工作。
- **任意精度类型**：ap_int、ap_fixed，直接对应量化后的位宽。
- **接口**：m_axi 访问 DDR，s_axilite 配参数，axis 走数据流。
- 循环分块（tiling）与双缓冲（ping-pong）：大模型放不进片上存储时的标准做法。

动手任务

- [ ] 跑通 Vitis-HLS-Introductory-Examples 里的 pipeline、array partition、dataflow 示例　1 小时可做　仿真

  先看官方示例里每个 pragma 改变了什么，再自己写。

  1. git clone https://github.com/Xilinx/Vitis-HLS-Introductory-Examples，切到和 2023.1 对应的分支。
  2. 依次跑 Pipelining 目录下的循环流水线示例、Array 目录下的 array_partition 示例、Task_level_Parallelism 目录下的 dataflow 示例，命令是 vitis_hls -f run_hls.tcl。
  3. 每个示例打开综合报告，看 Latency、II 和资源；把 pragma 删掉再跑一次对比。

  做完的标志　能用自己的话说清 pipeline、array partition、dataflow 各解决什么问题。

- [ ] 写 INT8 GEMM：从朴素三重循环开始，逐步优化到 II=1，每一步记录延迟和资源　1 小时可做　仿真

  GEMM 是这条路线的核心算子。从最慢的写法一步步优化，体会每个 pragma 的效果和代价。

  1. 写 gemm(int8 A[32][32], int8 B[32][32], int32 C[32][32])，朴素三重循环；C++ 测试主函数用软件参考结果比对。
  2. csim 通过后 csynth，记下延迟、II、DSP、BRAM、LUT。
  3. 给最内层循环加 PIPELINE；II 大于 1 时看报告给出的原因：内存端口不够还是循环依赖。
  4. 用 ARRAY_PARTITION 解决端口不够；调整循环顺序或累加方式，消除循环依赖。
  5. 每改一步记一行，直到 II = 1。

  做完的标志　一张至少 5 行的优化表，最后一行 II = 1，每一行都能解释为什么变了。

- [ ] 写 3×3 卷积 kernel（line buffer + 滑窗），通道数可配置　仿真

  卷积是 CNN 的主体。line buffer 让每个像素只从外部读一次，是 FPGA 上做卷积的标准结构。

  1. 先写一个朴素版本当参考：整张图放在数组里直接算。
  2. 再写流式版本：像素逐个输入，用两行 line buffer 加一个 3×3 窗口寄存器。
  3. 通道数做成模板参数或者循环。
  4. cosim 对比两个版本的输出，再比较资源用量。

  做完的标志　流式版本做到每拍处理一个像素（II = 1），输出和朴素版本一致。

- [ ] 给 GEMM 加上 m_axi 接口和分块，模拟从 DDR 搬数据　仿真

  真实的矩阵放不进片上存储，必须分块从 DDR 搬进来算。这是后面所有加速器都要面对的问题。

  1. 接口改成 m_axi（A、B、C 放在 DDR 里），控制用 s_axilite。
  2. 分块：每次把 A、B 的一块读进片上 buffer，算完再把 C 写回。
  3. 改变块大小，观察延迟和 burst 长度的变化。
  4. 在 cosim 波形里看 AXI 读写事务。

  做完的标志　256×256 矩阵结果正确；有一张块大小和延迟的关系表。

- [ ] 实现一种稀疏 GEMM（如 2:4 结构化稀疏），和稠密版本对比资源与周期　仿真

  2:4 稀疏能让乘法器减半。这是 S6 稀疏加速的预演。

  1. 2:4 的意思是每 4 个连续权重里最多 2 个非零，只存非零值和 2 位索引。
  2. 写压缩函数（Python 或 C++），把稠密权重转成值加索引的格式。
  3. HLS 里按索引挑出对应的激活值做乘加。
  4. 和稠密版本对比资源和周期。

  做完的标志　结果和稠密版本一致，有一张资源和周期的对比表。

- [ ] 把 GEMM IP 接到 PS 上，用裸机程序驱动，在 FZ5D 上比对结果并测实际周期数　需板

  把 HLS 加速器真正接进系统，在板上由 CPU 调用。这是 S5、S6 部署链路的雏形。

  1. 在 HLS 里 Export RTL 成 IP，加进 S2 的 FZ5D 工程。
  2. m_axi 接到 PS 的 S_AXI_HP0_FPD，s_axilite 接到 M_AXI_HPM0_FPD，跑 Connection Automation。
  3. Vitis 裸机程序：在 DDR 里准备 A、B，用 HLS 生成的驱动函数启动加速器，等它完成后比对 C。
  4. 注意 cache：启动前用 Xil_DCacheFlushRange 把输入刷到 DDR，读结果前用 Xil_DCacheInvalidateRange 让 cache 失效。
  5. 用 XTime 计时，和 HLS 报告的延迟对比。

  做完的标志　板上结果正确，实测周期和 HLS 报告的差距能解释清楚。

- [ ] 同一份 HLS 代码把目标器件换成 XCZU4EV 再综合，对比延迟、II 和资源，看哪些 pragma 的效果跟器件有关　1 小时可做　仿真

  同一份 HLS 代码换一颗芯片，看哪些结果和器件有关。

  1. 在同一个工程里新建一个 solution，器件选 XCZU4EV，其余设置不变。
  2. 跑 csynth，对比延迟、II、资源和估计频率。

  做完的标志　一张两列的对比表，说明哪些 pragma 的效果随器件变化。

> **验收标准**
> 一张优化表：同一个 GEMM 在不同 pragma 下的周期数、DSP/BRAM 用量和频率。

和 AI 助手的分工　预计省约 10 小时

自己动手

- 每一步 pragma 改动自己做，并解释 II 和资源为什么变了
- 稀疏 GEMM 的数据流设计

交给 AI，你来验收

- csim → 综合 → cosim → 导出 IP 的 Tcl 流程脚本
- pragma 组合的批量扫描和结果汇总
- testbench 用的 golden 数据生成

别做或少做

- 一组一组手动跑 pragma 实验

学习资料

- Kastner 等《Parallel Programming for FPGAs》 https://arxiv.org/abs/1805.03648
  免费，HLS 入门最好的教材
- pp4fpgas-cn：上一本书的中文翻译 https://xupsh.gitbook.io/pp4fpgas-cn/ 中文
  配套 HLS 工程在 GitHub 的 xupsh/pp4fpgas-cn-hls。译稿不一定跟上原书最新版，有疑问时对照英文
- AMD UG1399 Vitis HLS 用户指南
  pragma 的权威说明
- Xilinx/Vitis-HLS-Introductory-Examples https://github.com/Xilinx/Vitis-HLS-Introductory-Examples
  小而全的示例集
- Xilinx/Vitis-Tutorials https://github.com/Xilinx/Vitis-Tutorials
  官方教程合集

容易踩的坑

- 用了动态内存、递归、STL 容器，这些不可综合。
- csim 通过就以为对了。必须过 cosim。
- 盲目 UNROLL，资源直接爆掉。

待讨论

阶段 4

AI 加速器体系结构（可与 S3 并行）

约 60 小时　全部可在仿真中完成　硬件　编译

能估算一个模型在某块板子上的性能上限，并设计合理的计算阵列和存储层次。

这一阶段区分“会用工具”和“会设计加速器”。稀疏和低比特的收益能不能兑现，取决于这里的设计。

要掌握的概念

- **Roofline 模型**：算力上限、带宽上限和算术强度，判断一层是算力受限还是带宽受限。
- **数据复用方式**：权重固定、输出固定、行固定（Eyeriss）。
- **两种架构**：逐层流水的数据流架构（FINN）与通用覆盖层（DPU 一类 overlay）。
- 片上缓冲规划与 tiling 参数搜索。
- **量化算子**：INT8×INT8→INT32 累加，再用乘法加移位做 requantize。
- **量化的选择**：INT8 最常见，先用它跑通；之后可以试 INT4 或混合精度、FP8 或 MX 块浮点、2 的幂量化。位宽越低，一个 DSP 能做的乘法越多，权重带宽也越省。
- **稀疏**：非结构化稀疏很难在硬件上兑现加速，N:M 和块稀疏更友好。
- **压缩格式**：CSR、CSC、bitmask，以及索引带来的额外存储和带宽开销。

动手任务

- [ ] 用 Python 脚本统计 ResNet-18 每层的 MAC 数、参数量和激活大小　1 小时可做　仿真
- [ ] 以主力板 FZ5D 的 XCZU5EV 为目标（1248 DSP、BRAM 5.1 Mb + URAM 18 Mb、DDR4 峰值约 19.2 GB/s）画 Roofline，标出每层的位置；再把 XCZU4EV（728 DSP）的算力上限也画上去，看哪些层会因为换板而改变瓶颈　1 小时可做　仿真
- [ ] 精读 Zhang et al. FPGA'15 和 Eyeriss，用自己的话复述 tiling 和数据流设计　1 小时可做　仿真
- [ ] 读 EIE，搞清楚稀疏压缩格式在硬件上怎么解码　1 小时可做　仿真
- [ ] 按页面下方的“量化与数据格式约定”定下自己的默认格式（先全部用默认值）　1 小时可做　仿真

> **验收标准**
> 一份针对 FZ5D 的加速器设计文档：阵列规模、存储规划、各层预估延迟。阵列规模写成参数，同一份设计按 728 DSP 缩小就能放进 AXU4EVB-E。这是进入模型部署前的最后一个里程碑。

和 AI 助手的分工　预计省约 6 小时

自己动手

- 精读论文，做出设计文档里的阵列规模和存储划分决策

交给 AI，你来验收

- 每层 MAC、参数量、激活大小的统计脚本，Roofline 画图脚本
- 陪读论文、解释术语（关键数字回原文核对）

别做或少做

- 手算每一层的 MAC

学习资料

- Zhang et al., Optimizing FPGA-based Accelerator Design for Deep CNNs, FPGA 2015
  Roofline + tiling 的经典范例
- Chen et al., Eyeriss, ISCA 2016
  数据复用与行固定数据流
- Han et al., EIE, ISCA 2016
  稀疏压缩模型的硬件推理
- Williams et al., Roofline, CACM 2009
  性能上限分析方法

容易踩的坑

- 只算峰值算力，忽略带宽。
- 稀疏率 90% 不等于加速 10 倍，索引开销和负载不均会吃掉大部分收益。

待讨论

- S6 自己做 INT8 量化时，用 PyTorch 自带的量化工具还是 Vitis AI 量化器？前者通用、方便导出给自研加速器，后者的结果能直接给 DPU 跑。

阶段 5

嵌入式 Linux 与系统集成

约 46 小时　模型上板阶段，其中 1 项任务可先仿真　运行时　硬件

在板上跑起 Linux，从用户态调用自己的加速器。系统镜像用最省事的办法拿到，精力放在“运行时怎么加载和调用硬件”上。

模型部署要在 Linux 下进行，但系统镜像只是手段。PetaLinux 已被 AMD 宣布退役（2026 年冬季发最后一版），编译又慢又吃内存和磁盘，不值得深挖。真正要掌握的是启动链、设备树、驱动、DMA 和 cache 一致性，这些换任何镜像构建工具都通用。

要掌握的概念

- **系统镜像的几种来路，选一种能用的即可**：① 厂家资料包里的现成镜像，只替换比特流和设备树（FZ5D 的米尔镜像是 2019.1，太旧，只适合用来验硬件，和 2023.1 的驱动、运行时对不上）；② 用 S2 导出的 XSA 编一次 PetaLinux；③ 直接用 Yocto + meta-xilinx（PetaLinux 底下就是它，也是 AMD 新框架 EDF 的基础）；④ Debian / Ubuntu 根文件系统配 Xilinx 内核，能直接 apt、pip 装软件；⑤ Buildroot，最轻量。
- **启动链**：FSBL、PMU 固件、ATF、U-Boot、内核、根文件系统各自做什么，BOOT.BIN 里装了什么。不管用哪种工具构建，启动链都一样。
- **嵌入式 Linux 基础**：交叉编译、设备树、驱动加载。Zynq 的 ARM 端跑的是 Linux，这部分比单片机知识更有用。
- **运行时加载比特流**：fpgautil 或设备树 overlay。系统镜像做好一次，之后改硬件只需重新生成比特流和设备树 overlay。
- **用户态访问硬件**：UIO 或 /dev/mem 读写寄存器；XRT 加 Vitis 嵌入式流程是更正式的软件栈。
- **DMA 与 cache 一致性**：最常见的“结果偶尔不对”的原因。
- **裸机也是选项**：不需要 Linux 的实验不必上 Linux。S9 参考的 LLM 论文就是为了把内存全留给模型而裸机运行。
- **PYNQ**：用 Python 直接操作硬件很方便，但官方不支持这块板，要自己移植镜像，可以放到选做。

动手任务

- [ ] 在 FZ5D 上拿到一个能启动的 2023.1 Linux：用 S2 导出的 XSA 在服务器上编一次 PetaLinux 或 Yocto 镜像，从 TF 卡启动。能通过网口 SSH 登录就停　需板
- [ ] 画一张启动链示意图：BOOT.BIN 里每个组件干什么，比特流和设备树分别在哪一步加载　1 小时可做　仿真
- [ ] 对照：在 AXU4EVB-E 上用 ALINX 的现成镜像（没有就用同样的流程自己编）跑同一个寄存器读写脚本，比较“厂家给成熟 BSP”和“自己从 XSA 建镜像”两条路的工作量，以及两块板设备树的差别　需板
- [ ] 在 Linux 下用 fpgautil 或设备树 overlay 运行时加载自己的比特流，从用户态读写 S2 的 AXI-Lite 寄存器　需板
- [ ] 把 S3 的 HLS GEMM 通过 XRT 或 DMA 驱动调用，Python 喂数据，结果与 NumPy 比对　需板
- [ ] 测实际吞吐与理论值的差距，定位是计算、DMA 还是软件开销　需板
- [ ] 选做，替代技术任选一个深入：Yocto + meta-xilinx 直接构建镜像；做一个 Debian / Ubuntu 根文件系统；AMD 新的 EDF（需要 2025.1 以后的工具，留到升级版本时再试）　需板
- [ ] 选做：给这块板移植 PYNQ（FZ5D 和 KV260 是同一颗芯片，可以参考 KV260 的 PYNQ 镜像）　需板

> **验收标准**
> 一个在板上 Linux 里运行的 Python 脚本：调用自己写的 GEMM 加速器，给出正确性和实测带宽、吞吐。

和 AI 助手的分工　预计省约 10 小时

自己动手

- 弄懂启动链、设备树里自己 IP 的那几行
- cache 一致性问题的根因

交给 AI，你来验收

- 镜像构建的配置和报错排查（PetaLinux、Yocto 都一样，几千行编译日志交给它读）
- 设备树 overlay、UIO / XRT 调用的 Python 样板
- 在板上自动跑测试、收集日志的脚本

别做或少做

- 为了“学全”把几种镜像构建工具都走一遍
- 从零写 Linux 内核驱动（UIO / XRT 够用）
- 没有明确需要时移植 PYNQ

学习资料

- ALINX 给 AXU4EVB-E 配套的 Linux 教程　中文
  PetaLinux 的操作步骤写得细，换到 FZ5D 上照着做，只是把 BSP 换成自己导出的 XSA
- Bootlin 嵌入式 Linux 培训资料 https://bootlin.com/training/embedded-linux/
  免费公开的讲义，启动链、设备树、交叉编译讲得清楚，不绑定任何构建工具
- Yocto Project 文档 https://docs.yoctoproject.org
  走 Yocto 或以后换 EDF 时看
- Xilinx/meta-xilinx https://github.com/Xilinx/meta-xilinx
  AMD 器件的 Yocto 层，PetaLinux 和 EDF 底下都是它
- AMD Embedded Development Framework（EDF） https://www.amd.com/en/support/downloads/adaptive-socs-and-fpgas/embedded-software.html
  PetaLinux 的官方替代，含退役时间表
- AMD UG1144 PetaLinux Tools Reference Guide
  只在走 PetaLinux 时查
- PYNQ https://www.pynq.io
  选做移植时参考

容易踩的坑

- 在镜像构建上花掉大部分时间。目标是能 SSH 登录、能加载自己的比特流，镜像能用就停，别陷进 Yocto 配置里。
- 没有刷 cache，读到旧数据。
- 比特流、设备树和驱动版本不匹配。最典型的是用米尔的 2019.1 镜像去加载 2023.1 生成的比特流和驱动。
- PetaLinux / Yocto 编译要 100 GB 以上磁盘、16 GB 以上内存，第一次还要下载大量软件包。放到服务器上跑，别占台式机。

待讨论

- FZ5D 从 TF 卡还是 eMMC 启动？启动模式拨码开关在哪？（查米尔的硬件手册）
- 板子放在哪里、能否远程访问（串口、网口）？实验室有没有现成的镜像可以直接用？

阶段 6

分类模型

约 150 小时　模型上板阶段，其中 4 项任务可先仿真　编译　运行时　算子

在 FPGA 上跑通分类模型。三条路线各走一遍，最终用自己的加速器跑量化稀疏版 ResNet。

分类模型结构规整，是验证整条部署链路最好的起点。这里搭好的逐层比对和测量方法，后面每一站都会复用。

要掌握的概念

- **端到端流程**：量化模型 → 图解析（ONNX / QONNX）→ 算子映射 → 权重打包 → 硬件执行 → 后处理。
- **层融合**：conv + bn + relu 合并成一个硬件算子。
- **残差连接**：需要额外缓存一路激活。
- **验收指标**：Top-1 精度损失、延迟、FPS、功耗、FPS/W、资源利用率。

动手任务

- [ ] 用 PyTorch 自带量化工具把 ResNet-18 做 INT8 训练后量化，导出整数权重、scale 和每层 golden 输出　仿真
- [ ] 用 hls4ml 把 MNIST MLP 转成 HLS 工程，完成 csim 和 cosim　仿真
- [ ] 跑通 FINN 官方的 CIFAR-10 端到端示例（CNV 网络）　仿真
- [ ] 用自写加速器跑 LeNet 或小 CNN，逐层与 Python golden 比对　仿真
- [ ] 把 DPU（DPUCZDX8G）集成进 FZ5D 的硬件平台，参考 KV260（同为 XCZU5EV）的 DPU 设计配单核 B4096，用 Vitis AI 跑 ResNet-50 作为性能基线；有余力时在 AXU4EVB-E 上放 B1600–B2304 做对照，得到一张吞吐随 DSP 数量变化的表　需板
- [ ] 用自写加速器跑 INT8 ResNet-18，再用幅值剪枝做一版 2:4 稀疏，对比稠密和稀疏的实测延迟（量化同学的模型到了再替换）　需板
- [ ] 测功耗（板载传感器或功率计），算出能效　1 小时可做　需板

验收标准

一张对比表：DPU、FINN、自研加速器在同一模型上的精度、延迟、能效和资源；自研加速器一栏包含 INT8 稠密和 2:4 稀疏两个版本。

可投方向　第一篇论文可以从这里出：投 FPGA 或 DATE（B 类）；协同设计的故事讲得好就冲 DAC（A 类），之后扩展投 TCAD。　投稿目标

和 AI 助手的分工　预计省约 30 小时

自己动手

- 自研加速器的架构和关键模块
- 逐层误差定位时的判断：问题出在量化、打包还是硬件

交给 AI，你来验收

- PTQ 脚本、权重导出打包、逐层 golden 比对框架
- DPU 集成的 Tcl 和 PetaLinux 配置，照 ALINX 教程和 PG338 走（版本你来定）
- 实验结果表格和图的自动生成

别做或少做

- 自己研究量化算法（用标准 PTQ，等同学的成果）
- 手写 ONNX 解析器（用现成库）

学习资料

- hls4ml https://fastmachinelearning.org/hls4ml/
  Keras / PyTorch / QONNX 转 HLS
- FINN https://xilinx.github.io/finn/
  配合 Brevitas 量化训练的数据流编译器
- Umuroglu et al., FINN, FPGA 2017
  数据流架构的代表论文
- Vitis AI Model Zoo
  现成的量化模型，做基线用
- AMD PG338 DPUCZDX8G 产品手册
  DPU 各配置的资源占用和集成流程，选配置时查
- Kria KV260 的 Vitis AI / DPU 参考设计
  和 FZ5D 是同一颗芯片，DPU 配置、时钟和资源占用基本能照搬，只换 PS 配置和引脚
- ALINX 的 MPSoC Vitis AI 教程　中文
  中文步骤，做 AXU4EVB-E 对照时用
- PyTorch 量化文档 https://pytorch.org/docs/stable/quantization.html
  训练后量化（PTQ）的标准做法，自己量化模型时用

容易踩的坑

- 不做逐层 golden 比对，最后输出错了无从查起。
- 两块板都不在 Vitis AI 的官方支持板卡里，DPU 要自己集成进硬件平台；版本必须和 Vivado、PetaLinux 对齐。FZ5D 先照 KV260 的参考设计改。
- DPU 的板上运行时（VART、XRT）最省事的装法是 PetaLinux / Yocto 的 meta-vitis-ai 层。S5 如果用的是现成镜像，走 DPU 时才需要认真编一次 PetaLinux 或 Yocto 镜像，编译放服务器。

待讨论

- Vitis AI DPU 只作为基线，还是也考虑作为最终方案的一部分?

阶段 7

检测与分割模型

约 160 小时　模型上板阶段，其中 4 项任务可先仿真　算子　运行时

部署轻量 YOLO 检测模型，完成从摄像头输入到画框输出的实时系统；再部署语义分割和实例分割模型。

检测比分类多出大分辨率输入、多尺度输出、上采样和拼接算子，以及复杂的后处理。分割又多一个解码器：要把特征图放大回原图分辨率，编码器的中间结果要一直存到解码器用到为止，输出是逐像素的。两者合起来，考验的是软硬件划分和存储带宽规划。

要掌握的概念

- **轻量检测结构**：YOLOv5n、YOLOv8n 一类，以及其中的上采样、concat、多分支。
- **检测后处理**：框解码和 NMS 一般放在 ARM 端。
- **编码器-解码器与跳跃连接**：UNet 一类结构里，编码器每一级的输出要缓存到解码器对应层，片上放不下就得进出 DDR。
- **上采样算子**：转置卷积、双线性插值、最近邻插值，硬件实现代价差别很大。
- **空洞卷积**：DeepLab 系列常用，访存模式和普通卷积不同。
- **实例分割**：YOLOv8-seg 输出 mask 原型和系数，mask 合成可以放 ARM 端，也可以做成一个小矩阵乘放进 FPGA。
- **软硬件流水**：预处理、推理、后处理三段重叠执行。
- **视频通路**：MIPI 摄像头 → 预处理 → 推理 → 显示。
- **量化敏感度**：INT8 对 mAP 和 mIoU 的影响通常比对分类精度更明显，分割的边缘细节尤其容易受影响。

动手任务

- [ ] 统计 YOLO 小模型各层计算量，用 S4 的方法评估能否放下　1 小时可做　仿真
- [ ] 扩展自研加速器：支持上采样、concat、stride-2 卷积　仿真
- [ ] 统计 UNet 各级跳跃连接需要缓存的激活大小，判断哪些能留在片上、哪些必须进 DDR　1 小时可做　仿真
- [ ] 扩展加速器：支持转置卷积或双线性上采样、空洞卷积，cosim 验证　仿真
- [ ] 用 S6 集成好的 DPU 在板上跑 Vitis AI 的 YOLO 示例　需板
- [ ] backbone 用自研加速器，后处理放 ARM，跑通单张图片检测　需板
- [ ] 在 FZ5D 上接 USB 摄像头做实时检测（它有 USB 3.0 和 HDMI 输入），测端到端延迟和 FPS；AXU4EVB-E 的 MIPI 接口能用的话，配 ALINX 摄像头模组做一次对照，体会不同视频输入链路对延迟的影响　需板
- [ ] 部署轻量语义分割模型（如小型 UNet，或 MobileNet 骨干的 DeepLabv3+），测 mIoU 和 FPS　需板
- [ ] 选做：部署 YOLOv8n-seg 实例分割，比较 mask 合成放 ARM 和放 FPGA 的延迟　需板

> **验收标准**
> 实时检测 demo 和语义分割 demo，各附一份延迟分解（预处理、推理、后处理），以及分割模型的 DDR 带宽占用分析。

可投方向　检测、分割的软硬件划分和带宽优化：DAC、ICCAD、DATE、FPGA；完整的实时端侧系统也可以考虑 MobiCom、UbiComp。　投稿目标

和 AI 助手的分工　预计省约 30 小时

自己动手

- 软硬件怎么划分
- 新算子的硬件结构，带宽分析的结论

交给 AI，你来验收

- 预处理和后处理（letterbox、NMS、分割上色）的 C++ 实现
- 摄像头采集和显示的 OpenCV / GStreamer 胶水代码
- mAP、mIoU 评测脚本

别做或少做

- 自己从头训练检测、分割模型（用预训练权重）

学习资料

- KV260 Smart Camera 加速应用
  AMD 官方的检测完整系统。不能直接在你的板子上运行，但视频通路和软件架构可以参考
- Vitis AI Model Zoo 中的检测与分割模型
  基线
- Ultralytics YOLO 文档 https://docs.ultralytics.com
  检测、实例分割模型的结构与 ONNX 导出
- Ronneberger et al., U-Net, MICCAI 2015
  编码器-解码器 + 跳跃连接的原型
- Chen et al., DeepLabv3+, ECCV 2018
  空洞卷积与轻量解码器

容易踩的坑

- 只优化推理，结果瓶颈在 Python 后处理或图像缩放。
- mAP 或 mIoU 下降找不到原因。需要逐层检查量化误差。
- 分割模型的跳跃连接激活反复进出 DDR，带宽成为瓶颈，计算阵列大部分时间在等数据。
- 高分辨率分割图的 argmax 和上色放在 Python 里做，比推理还慢。

待讨论

- 你们的应用更需要语义分割（每个像素分类）还是实例分割（区分每个物体）？输入分辨率大概多少？

阶段 8

Transformer 与多模态

约 160 小时　模型上板阶段，其中 2 项任务可先仿真　算子

让加速器支持 Transformer 核心算子，先部署 ViT 小模型，再部署图文多模态模型的视觉编码部分。

从 CNN 到 Transformer，计算主体从卷积变成大矩阵乘，同时多出 softmax、LayerNorm、GELU 等非线性算子。这是通往 LLM 的必经之路。

要掌握的概念

- **自注意力**：QKV 投影、QK^T、softmax、乘 V。
- **非线性算子硬件化**：查表、分段线性近似、整数近似（I-BERT 的做法）。
- **LayerNorm**：要对整行求均值和方差，需要归约和额外缓存。
- **动态矩阵乘**：QK^T 的两个操作数都是激活，和固定权重的矩阵乘在数据流上不同。
- **token 剪枝与稀疏注意力**：Transformer 上的稀疏化手段。
- **多模态**：视觉编码器（ViT）+ 投影层 + 语言模型，可以先只加速视觉编码器。

动手任务

- [ ] 用 HLS 实现定点版 softmax、LayerNorm、GELU，测与浮点的误差　仿真
- [ ] 实现多头注意力模块，在 cosim 中验证　仿真
- [ ] 部署 DeiT-Tiny 或 ViT-Tiny 分类，与 S6 的 CNN 比较效率　需板
- [ ] 部署 CLIP 类模型的图像编码器，文本侧先放 CPU　需板

> **验收标准**
> ViT-Tiny 在板上运行，给出各算子的延迟占比，说明非线性算子的开销。

可投方向　非线性算子硬件化、ViT 数据流：DAC、ICCAD、FPGA；有硬件感知的量化或剪枝方法时可以考虑 NeurIPS、CVPR。　投稿目标

和 AI 助手的分工　预计省约 20 小时

自己动手

- softmax、LayerNorm 的硬件近似方案
- 注意力的数据流设计

交给 AI，你来验收

- 定点近似的误差扫描脚本
- 从 Hugging Face 模型导出各层中间结果，作为 golden
- CLIP 文本侧的 CPU 推理

别做或少做

- 手写 tokenizer 和模型加载代码

学习资料

- Kim et al., I-BERT: Integer-only BERT Quantization, ICML 2021
  整数化 softmax / GELU / LayerNorm
- Dong et al., HeatViT, HPCA 2023
  FPGA 上的 ViT token 剪枝加速
- Dosovitskiy et al., ViT, ICLR 2021
  视觉 Transformer 原文

容易踩的坑

- softmax 直接用浮点 exp，资源暴涨。
- 忽略注意力矩阵随序列长度平方增长的存储需求。

待讨论

阶段 9

大语言模型

约 200 小时　模型上板阶段，其中 2 项任务可先仿真　运行时　算子

在主力板 FZ5D 上跑通 0.5B 级 LLM 的完整推理，INT4 解码做到 40 tok/s 以上（带宽利用率六成以上），并能说清剩下的差距在哪。

解码阶段每生成一个 token 都要把全部权重读一遍，瓶颈是内存带宽而不是算力。这恰好是稀疏和低比特量化在 FPGA 上最能发挥价值的地方。

要掌握的概念

- **Prefill 与 Decode**：前者计算受限，后者带宽受限，最好用不同的硬件策略。
- **KV Cache**：随生成长度增长，需要专门的存储管理。
- **混合精度**：权重 INT4、激活 INT8，group-wise 量化。
- **稀疏权重**：直接减少要搬运的字节数。
- **解码速度估算**：tok/s ≈ 实际可用带宽 ÷ 每个 token 要读的权重字节数。实测 tok/s 除以理论上限就是带宽利用率，是这一阶段最重要的指标。
- **存储**：HBM 与 DDR 的带宽差一个数量级。你的板还有一块 1 GB 的 PL 侧 DDR4（约 4.3 GB/s），可以放 KV Cache 或一部分权重，理论总带宽多出约两成。
- **主机协同**：PCIe、XRT；采样和 tokenizer 放在 CPU 端。

动手任务

- [ ] 用 Python 按“带宽 ÷ 每 token 权重字节”算出下方预期表里各模型的解码上限，后面每次实测都和它对照　1 小时可做　仿真
- [ ] 先用 llama.cpp 在电脑上跑通 Qwen2.5-0.5B 作为功能 golden，再在板上 A53 跑一遍作为 CPU 基线（预计 2–5 tok/s）　需板
- [ ] 实现带 KV Cache 的 decoder layer，cosim 验证单层　仿真
- [ ] 先在板上跑通极小模型（几十 M 参数的玩具 GPT）　需板
- [ ] 第一版：FPGA 加速矩阵乘，跑通 0.5B INT8 完整推理，目标 10–15 tok/s　需板
- [ ] 优化版：权重换成 INT4，读权重和计算重叠，突发访存对齐，非线性算子放进 FPGA，目标 40 tok/s 以上，并测能效　需板
- [ ] 加入稀疏权重解码，看带宽节省是否转化为速度提升　需板
- [ ] 选做挑战：在 FZ5D 上参照 KV260 的论文跑 7B INT4（约 5 tok/s），8 GB 内存可以在 Linux 下跑，不必像论文那样裸机；或者回到 AXU4EVB-E，把 KV Cache 放到它独有的 PL 侧 DDR；有 HBM 板卡时迁移过去　需板

验收标准

端到端 LLM 推理 demo 和带宽利用率报告：A53 基线、第一版、优化版三档 tok/s，对照下方预期表，分析剩下的差距在哪。

可投方向　嵌入式 FPGA 上的 LLM 解码优化：DAC、ICCAD、FPGA；有架构层面的新见解再考虑 HPCA、MICRO。期刊扩展投 TCAD、TC。　投稿目标

预期解码速度（tok/s）

模型与量化	每 token 读权重	理论上限	第一版	优化后	论文水平
Qwen2.5-0.5B INT8	约 0.5 GB	约 38	10–15	**约 25**	约 33
Qwen2.5-0.5B INT4	约 0.28 GB	约 69	20–25	**约 45**	约 60
Qwen2.5-1.5B INT4	约 0.87 GB	约 22	7–9	**约 14**	约 19
Llama2-7B INT4（挑战）	约 3.5 GB	约 5.5	—	—	4.9（KV260 实测）

按 PS 侧 DDR4 峰值 19.2 GB/s 估算；第一版、优化后、论文水平分别按带宽利用率约 30–40%、60–70%、85–93% 计算。KV260 的 PS 内存和你的两块板完全相同，论文结果可以直接对照。对照基线：A53 跑 llama.cpp 预计 2–5 tok/s。prefill 受算力限制，另算：728 个 DSP 的 INT8 峰值约 0.7 TOPS，0.5B 模型先按每秒 100–300 个 token 估计。

和 AI 助手的分工　预计省约 35 小时

自己动手

- decode 数据流和 DDR 访问模式设计
- 带宽利用率分析，决定下一步优化什么

交给 AI，你来验收

- llama.cpp 交叉编译到 A53，跑出 CPU 基线
- INT4 权重重排、打包工具
- tok/s 和带宽的测量脚本；采样、tokenizer、对话外壳等主机侧代码

别做或少做

- 自己从头写推理框架的外围（拿 llama.cpp 当外壳和 golden）

学习资料

- Li et al., Pushing up to the Limit of Memory Bandwidth and Capacity Utilization for Efficient LLM Decoding on Embedded FPGA, DATE 2025 https://arxiv.org/abs/2502.10659
  KV260 裸机跑 Llama2-7B 4-bit，4.9 tok/s，带宽利用率约 85%。内存和你的板相同，是最直接的参照
- Hummingbird: A Smaller and Faster LLM Accelerator on Embedded FPGA, 2025 https://arxiv.org/abs/2507.03308
  同在 KV260 上，分析 Zynq 内存控制器端口仲裁，带宽利用率做到 93–94%
- llama.cpp https://github.com/ggml-org/llama.cpp
  功能 golden 和 A53 上的 CPU 基线
- Zeng et al., FlightLLM, FPGA 2024
  稀疏 + 混合精度的 FPGA LLM 推理，和你的方向最接近
- Hong et al., DFX, MICRO 2022
  多 FPGA 的 GPT 推理
- Chen et al., Understanding the Potential of FPGA-Based Spatial Acceleration for LLM Inference, TRETS 2024
  LLM 在 FPGA 上的性能建模
- Allo https://github.com/cornell-zhang/allo
  上一篇论文配套的加速器设计语言

容易踩的坑

- 以为这块板只能跑很小的模型。7B 的 INT4 权重约 3.5 GB，FZ5D 的 8 GB 内存连同 Linux 和 KV Cache 一起放得下，真正的限制是带宽：速度和模型大小成反比。
- 只报 tok/s，不报带宽利用率。不和理论上限对照，就不知道还差多少、差在哪。
- prefill 和 decode 用同一套硬件策略。

待讨论

- 研究上更想做哪条：在 0.5B 上做稀疏、低比特的软硬件协同，还是挑战 7B 级模型在嵌入式 FPGA 上的极限？
- 想把速度提高一个数量级时，能否申请到 HBM 板卡（如 Alveo U280 / U55C）、Versal 平台或云 FPGA 实例？

可选支线

可选支线：GPU 与 AI infra 衔接

约 40 小时　　需要 NVIDIA GPU，不需要 FPGA 板；建议穿插在 S8–S9 期间　　算子　运行时

能写基本的 CUDA 和 Triton kernel，读懂主流推理引擎的核心机制，并能为论文跑出调优过的 GPU 基线。

FPGA 加速器论文几乎都要和 GPU 对比，GPU 基线没调好，结论就站不住。这些也是 AI infra 岗位的核心技能，而且和你在 FPGA 上学的 tiling、算子融合、量化一一对应，迁移成本很低。

要掌握的概念

- **GPU 执行模型**：线程、warp、block、SM。对照 FPGA：GPU 靠大量线程分时复用硬件，FPGA 靠把电路空间展开。
- **存储层次**：寄存器、shared memory、L2、HBM。shared memory 分块和你在 FPGA 上的 BRAM 分块是同一个思路。
- **Tensor Core 与低精度**：INT8、FP8，以及 Ampere 架构开始硬件支持的 2:4 结构化稀疏，可以直接和你的 FPGA 稀疏方案对比。
- **Triton**：用 Python 写 GPU kernel，比 CUDA 上手快，适合快速验证想法。
- **FlashAttention**：分块计算加在线 softmax，避免把整个注意力矩阵写回显存。
- **推理引擎机制**：llama.cpp 的量化格式和反量化 kernel；vLLM 的 PagedAttention 和连续批处理。
- **性能分析**：Nsight Systems 看整体时间线，Nsight Compute 看单个 kernel 的带宽和算力利用率。

动手任务

- [ ] 读 FlashAttention 论文，把它的分块思路和你在 S8 做的 FPGA 注意力实现对照　1 小时可做　只需电脑
- [ ] 读 llama.cpp 的量化格式和一个反量化 kernel 的实现　1 小时可做　只需电脑
- [ ] 写 CUDA 矩阵乘：从朴素版到 shared memory 分块，用 Nsight Compute 看带宽利用率　需 GPU
- [ ] 跟着 Triton 官方教程写 GEMM 和融合 softmax，和 CUDA 版本对比性能　需 GPU
- [ ] 用 vLLM 或 TensorRT-LLM 跑和 S9 相同的模型，测 tokens/s 和能效，作为 FPGA 方案的 GPU 基线　需 GPU
- [ ] 用 PyTorch 的 2:4 半结构化稀疏在 GPU 上跑一次推理，和你的 FPGA 稀疏方案对比　需 GPU

> **验收标准**
> 一张 GPU 与 FPGA 的对比表：同一模型的延迟、吞吐、功耗、能效，GPU 一侧附带 profiling 结果，证明基线已经调优。

和 AI 助手的分工　预计省约 10 小时

自己动手

- kernel 优化的思路：访存合并、shared memory、tiling

交给 AI，你来验收

- CUDA 环境、profiling（Nsight）命令和结果整理

别做或少做

- 系统学一遍 CUDA 全部 API

学习资料

- Kirk & Hwu《Programming Massively Parallel Processors》
  CUDA 入门的标准教材，有中文版
- NVIDIA CUDA C++ Programming Guide
  查阅用
- Triton 官方教程 https://triton-lang.org
  矩阵乘和融合 softmax 两篇最值得做
- Dao et al., FlashAttention, NeurIPS 2022
  分块思想在 GPU 上的代表作
- Kwon et al., vLLM / PagedAttention, SOSP 2023
  LLM 推理引擎的 KV Cache 管理
- llama.cpp https://github.com/ggml-org/llama.cpp
  量化格式和 CPU/GPU kernel 都值得读

容易踩的坑

- 拿没调优的 PyTorch eager 模式结果当 GPU 基线，审稿人会质疑对比不公平。
- 只比吞吐，不比功耗和批大小。FPGA 的优势通常在 batch=1 的低延迟和能效。
- 在支线上投入太多，挤占主线时间。按每周 7 小时，这条支线控制在 30–50 小时以内。

待讨论

- 实验室有没有可用的 NVIDIA GPU？型号是什么？2:4 稀疏需要 Ampere（如 A100、RTX 30 系）及以后的架构。

和 AI 编程助手协同

这里的 harness 指 Claude Code、Codex 这类能在你的工程目录里读写文件、跑命令的 AI 编程助手。学习过程中有大量重复、或者本来就不该由你来做的工作，交给它；要学的核心能力自己练，验收标准由你定。每个阶段都有一栏“和 AI 助手的分工”，按下面三类划分。照这个分工，主线预计能省约 **156** 小时。

自己动手

这条路线要你掌握的能力：核心 RTL 和 HLS、架构决策、调试时的判断、实验结论。AI 可以事后 review、解释，但不替你写第一版。

交给 AI，你来验收

重复性和胶水性的工作：环境搭建、构建脚本、测试激励、批量实验、日志排查、报表生成、主机侧外围代码。前提是有自动化的验收（golden 比对、一条命令的测试），否则省下的时间会在排查它的错误时还回去。

别做或少做

和目标无关、或者已有现成方案的事：手动重复操作、重复造轮子、超出需要的移植。

先搭好的基础设施

S1 起	每个工程用 git 管理，让 AI 改代码前先提交，改坏了能一键回滚。
S1 起	每个模块配一个 Python golden 和一条命令的自动比对（如 make test）。AI 改完自己跑，通过才算数，这是把活交出去的前提。
S2 起	Vivado 和 Vitis HLS 全部用 Tcl 脚本驱动（非工程模式）。GUI 里的点击 AI 做不了，脚本它能跑，报告它能读。
S2 起	在仓库根目录写一份项目说明（CLAUDE.md 或 AGENTS.md）：板卡型号、工具版本、目录结构、常用命令，以及不许 AI 改的文件（引脚约束、PS 配置）。
S2 起	可选：装 AMD Ross 的 Vivado MCP Server（2026 年 9 月发布，免费），让 AI 通过 Tcl 直接操作 Vivado，带 lint、时序方法学检查等技能，支持 Vivado 2020.2 及以后。它的 HLS 自动构建技能要求 2025.1 以上，2023.1 下不能用；S3 学 HLS 时 pragma 本来就该自己调，影响不大。
S2 起	远程构建：本机写代码、接板子烧写，综合、实现、HLS 和镜像编译放到服务器上跑。写一个脚本完成“同步代码 → 服务器上批处理构建 → 拉回比特流和报告”，交给 AI 维护。
S5 起	板子接网口，能 SSH 登录、拷文件、看串口日志，AI 就能替你跑板上测试。烧写 Flash、改启动配置这类操作由你确认后再做。
S6 起	实验结果统一写进 CSV 或 JSON，报告里的表和图由脚本生成，换模型、换配置后重跑一遍即可。

红线

- S1–S4 的核心 RTL 和 HLS 第一版自己写，写完再让 AI review，而不是它写你读。这部分能力就是你要学的东西。
- 引脚约束、电平标准、PS 时钟和 DDR 配置，对着板卡原理图（ALINX、米尔）亲自核对。写错了轻则不工作，重则可能损坏板上器件。
- AI 给出的工具选项、API 名称、论文数字，用之前查原文或实际跑一遍。Vivado、Vitis AI、XRT 各版本差异大，它常把不同版本的用法混在一起。
- AI 修好的 bug，要让它讲清根因，你能自己复述了再合并。时序问题、cache 一致性问题尤其如此。
- 报告和论文里的每个数字，都要能追溯到一次实际运行。

各阶段交给 AI 的部分

阶段	小时	可省	交给 AI 的工作
S0 全景认知	20	—	把 Vitis AI、FINN、hls4ml 的文档首页整理成对比表初稿，你对照原文核对
S1 数电与 Verilog	70	约 5	装 Icarus / Verilator / cocotb 的环境脚本和 Makefile；Python 随机激励和自动比对脚本；你写完 RTL 后让它 review：锁存器、阻塞赋值、复位遗漏
S2 FPGA 与 AXI	66	约 10	把工程改成 Vivado Tcl 脚本（非工程模式），一条命令重建；批量综合不同阵列规模，从报告里自动提取 DSP、频率，生成表格；裸机读写寄存器的 C 程序
S3 HLS 高层次综合	65	约 10	csim → 综合 → cosim → 导出 IP 的 Tcl 流程脚本；pragma 组合的批量扫描和结果汇总；testbench 用的 golden 数据生成
S4 加速器架构	60	约 6	每层 MAC、参数量、激活大小的统计脚本，Roofline 画图脚本；陪读论文、解释术语（关键数字回原文核对）
S5 Linux 与集成	46	约 10	镜像构建的配置和报错排查（PetaLinux、Yocto 都一样，几千行编译日志交给它读）；设备树 overlay、UIO / XRT 调用的 Python 样板；在板上自动跑测试、
S6 分类模型	150	约 30	PTQ 脚本、权重导出打包、逐层 golden 比对框架；DPU 集成的 Tcl 和 PetaLinux 配置，照 ALINX 教程和 PG338 走（版本你来定）；实验结果表格和图的自

阶段	小时	可省	交给 AI 的工作
S7 检测与分割	160	约 30	预处理和后处理（letterbox、NMS、分割上色）的 C++ 实现；摄像头采集和显示的 OpenCV / GStreamer 胶水代码；mAP、mIoU 评测脚本
S8 Transformer 与多模态	160	约 20	定点近似的误差扫描脚本；从 Hugging Face 模型导出各层中间结果，作为 golden；CLIP 文本侧的 CPU 推理
S9 大语言模型	200	约 35	llama.cpp 交叉编译到 A53，跑出 CPU 基线；INT4 权重重排、打包工具；tok/s 和带宽的测量脚本；采样、tokenizer、对话外壳等主机侧代码

这条路线在 AI 系统栈中的位置

AI infra 自上而下大致分六层。这条路线以最底下的硬件层为主体，向上覆盖算子、编译和运行时三层，可以算作 FPGA 版的 AI infra 下半部分。 每个阶段标题下的层次标签对应这里的分层，第一个是主要层次。下面每层列出的阶段里，加粗的是以该层为主的阶段。

集群与调度
不涉及

不涉及。

GPU 世界的对应：Kubernetes、训练集群调度

分布式训练
不涉及

不涉及。这条路线做推理，不做训练。

GPU 世界的对应：Megatron-LM、DeepSpeed、NCCL

推理引擎与运行时
部分覆盖

主机和加速器协同、前后处理流水、DMA 调度、KV Cache、prefill 与 decode 的调度。

GPU 世界的对应：vLLM、TensorRT-LLM、llama.cpp

S5 Linux 与集成 S6 分类模型 S7 检测与分割 **S9 大语言模型** CUDA 支线

AI 编译器
部分覆盖

把模型图映射到硬件：图解析、算子映射、层融合、tiling 参数、权重打包。hls4ml、FINN、Allo 都属于这一层。

GPU 世界的对应：TVM、MLIR、Triton

S4 加速器架构 **S6 分类模型**

算子与 kernel
覆盖

单个算子的高效实现。GPU 上写 kernel，你在 FPGA 上用 HLS 写电路。

GPU 世界的对应：CUDA kernel、FlashAttention

S3 HLS 高层次综合 S6 分类模型 **S7 检测与分割** **S8 Transformer 与多模态** S9 大语言模型 **CUDA 支线**

硬件与加速器架构
覆盖

计算阵列、片上存储层次、数据流设计，学术上属于计算机体系结构。这是路线的主体。

GPU 世界的对应：GPU、TPU、各类 AI 芯片

S1 数电与 Verilog **S2 FPGA 与 AXI** S3 HLS 高层次综合 **S4 加速器架构** S5 Linux 与集成

投稿目标

这条路线做的是 FPGA 上的端侧 AI 加速，偏软硬件协同设计，主战场是 CCF“计算机体系结构 / 并行与分布计算 / 存储系统”一类，其中最对口的 A 类会议是 DAC。 等级按 ccfddl 开源数据核对（2026 年 11 月）。各校认定的 CCF 目录版本和毕业要求不同，先和导师确认。

等级	会议或期刊	匹配度	说明
体系结构与 EDA（主战场）			
CCF A 会议	**DAC**	★★★	最对口的 A 类会议：软硬件协同、FPGA 加速器、量化稀疏的硬件实现都收
CCF A 会议	**ISCA、MICRO、HPCA、ASPLOS**	★★☆	体系结构四大顶会。FPGA 论文也有（如 MICRO'22 的 DFX），但需要架构层面的新见解
CCF A 期刊	**IEEE TCAD、IEEE TC、IEEE TPDS、ACM TACO**	★★★	会议论文扩展成期刊的主要去处
CCF A 期刊	**ESWEEK 期刊轨（CODES+ISSS、CASES、EMSOFT）**	★★★	录用后发在 TCAD 特刊，按 TCAD 计；具体规则以当年征稿说明为准
CCF B 会议	**FPGA（ACM/SIGDA）、ICCAD、DATE、ICCD**	★★★	FPGA 会议是 FPGA 圈公认最好的会，CCF 只给 B；对第一篇论文友好
CCF C 会议	**FCCM、FPL、FPT、ASP-DAC**	★★☆	FCCM 在 FPGA 圈认可度不低（FPL 的等级未在数据中核对到，按经验为 C）
端侧系统（移动与普适计算）			
CCF A 会议	**MobiCom、UbiComp（IMWUT）**	★☆☆	讲完整端侧 AI 系统时考虑，纯加速器论文少见
CCF B 会议	**MobiSys、SenSys**	★☆☆	同上，偏系统与应用
算法（量化、稀疏、高效模型）			
CCF A 会议	**NeurIPS、ICML、CVPR、ICCV**	★☆☆	需要算法层面的贡献；硬件感知的量化或稀疏方法加 FPGA 实测可以考虑
未收录 会议	**MLSys**	★★☆	不在 CCF 目录里，但 AI 系统圈认可度很高

按路线产出安排投稿

时间点	可能的论文	建议目标
S6–S7 之后	自研加速器 + INT8 / 2:4 稀疏，与 DPU、FINN 对比；某个算子或数据流上的协同设计	第一篇投 FPGA 或 DATE；故事好就冲 DAC；扩展投 TCAD
S8 之后	非线性算子硬件化、ViT 在嵌入式 FPGA 上的数据流	DAC、ICCAD、FPGA
S9 之后	嵌入式 FPGA 上的 LLM 解码：带宽利用率、把 PL 侧 DDR 当额外通道、稀疏 + 低比特	DAC、ICCAD、FPGA；有架构新见解再考虑 HPCA、MICRO
两三篇之后	更通用的加速器架构方法	ISCA、MICRO、HPCA、ASPLOS

路线里读过的论文发在哪

Zhang et al.（Roofline + tiling） FPGA'15 CCF B

FINN FPGA'17 CCF B

FlightLLM FPGA'24 CCF B

Li et al.（KV260 跑 7B） DATE'25 CCF B

EIE ISCA'16 CCF A

DFX MICRO'22 CCF A

你的板卡：EdgeBoard FZ5D 主力，AXU4EVB-E 对照

主线在 FZ5D 上走；每个阶段挑一件事在 AXU4EVB-E 上再做一遍，体会换一个硬件环境要改什么、性能怎么变。 两块板芯片同代、PS 侧内存带宽相同，差别在 DSP 数量、内存容量、有没有 PL 侧 DDR、厂家资料和接口，每一项差别都对应一个能做的对照实验。

主力板：EdgeBoard FZ5D（实验室库存，你独占）

米尔电子生产、百度飞桨生态的端侧 AI 计算卡，FZ5 系列的 8 GB 内存版本。实验室库存，你可以独占，作为整条路线的主力板。参数来自米尔的产品页和产品资料。

芯片	XCZU5EV（Zynq UltraScale+ MPSoC，EV 系列，和 KV260 用的 K26 同级）
处理器（PS）	4 核 Cortex-A53 1.5 GHz + 2 核 Cortex-R5
DSP	1248 个，约为 AXU4EVB-E 的 1.7 倍
片上存储	BRAM 5.1 Mb + URAM 18 Mb
内存	8 GB DDR4，64 位，2400 Mbps，峰值约 19.2 GB/s，全部在 PS 侧，没有 PL 侧独立 DDR
存储	32 GB eMMC，64 MB QSPI Flash，TF 卡
视频编解码（VCU）	有，和 AXU4EVB-E 一样
接口	千兆网口、4 个 USB 3.0、DP、HDMI 输入、USB 转串口、JTAG、RS232 / RS485、CAN（米尔产品页）
出厂软件	百度 EdgeBoard 系统：Paddle Lite 加百度自己的 FPGA 加速核，官方给的 ResNet-50 约 55 FPS；BSP 是 PetaLinux 2019.1

和 AXU4EVB-E 比

- DSP 多七成，DPU 能放下单核 B4096（AXU4EVB-E 是 B1600–B2304），同一个卷积网络的吞吐大约高 1.7–2 倍。
- 内存 8 GB，带宽和 AXU4EVB-E 相同。LLM 解码速度只看带宽，同一个模型的 tok/s 基本一样；多出来的 4 GB 让 7B INT4 模型能和 Linux、KV Cache 一起放进内存，不必裸机运行。
- 没有 PL 侧独立 DDR。PL 侧 DDR 是直接接在 FPGA 引脚上、给加速器独占的第二块内存，AXU4EVB-E 有一块 1 GB 的，但只有 4.3 GB/s，不能用来提速。S9 里“KV Cache 放 PL DDR”这个选做实验只能回到 AXU4EVB-E 上做，主线不受影响。
- 资料比 ALINX 少：出厂系统是百度的一套，和这条路线的 Vivado 流程无关，也没有中文的 Vivado 入门教程。要用自己的加速器，得用 Vivado 2023.1 新建硬件平台。先在米尔资料包里找 Vivado 工程、HDF 或 XSA，里面有 PS 侧 DDR 和引脚配置；有它就能导入新版本升级，没有的话 PS 配置要自己从原理图推，工作量大很多。所以 S1 的第一个上板实验先在 AXU4EVB-E 上照 ALINX 教程跑通，再移植过来。
- 两块板芯片同代、内存带宽相同，只有 DSP 数量和内存容量不同。正好把“算力”和“带宽”两个变量分开，适合做 roofline 验证和设计空间探索的实验。

对照板：ALINX AXU4EVB-E V3.0

2026 年 11 月到货，当时在 S1。核心板参数来自 ALINX 的 ACU4EV 核心板手册，芯片资源来自 AMD 的选型资料； 载板上的接口（MIPI、HDMI/DP 等）我没能读到这块板的手册原文，请以 ALINX 官方手册为准。

芯片	XCZU4EV-1SFVC784I（Zynq UltraScale+ MPSoC，EV 系列）
处理器（PS）	4 核 Cortex-A53 + 2 核 Cortex-R5
DSP	728 个，决定乘加算力的上限
片上存储	BRAM 4.5 Mb + URAM 13.5 Mb
PS 侧内存	4 GB DDR4，64 位，2400 Mbps，峰值约 19.2 GB/s
PL 侧内存	1 GB DDR4，16 位，2132 Mbps，峰值约 4.3 GB/s，可以给加速器单独用
存储	8 GB eMMC，256 Mb QSPI Flash
视频编解码（VCU）	EV 系列自带 H.264 / H.265 硬件编解码，做视频检测时可以用来推流
载板接口	MIPI、HDMI/DP、网口等请以 ALINX 官方手册为准（我没能读到这块板的手册原文）

和 KV260 比

- DSP 728 个，约为 KV260（1248 个）的六成，同等设计下能放的计算阵列小一些。
- 片上存储 18 Mb，KV260 约 23 Mb，相差不大。
- PS 侧都是 4 GB 64 位 DDR4，带宽相同，所以 KV260 上的 LLM 论文结果可以直接参照；你的板子还多一块 1 GB 的 PL 侧独立 DDR4。
- 不在 Vitis AI、PYNQ 的官方支持列表里：DPU 要自己集成，PYNQ 要自己移植。这是用 ALINX 板最主要的额外工作量，已经算进 S5、S6 的小时数。

这块板能走到哪一步

S1–S5	全部可以在这块板上完成
S6 分类	可以。DPU 用中等配置，自研加速器按 728 DSP 规划
S7 检测与分割	可以。轻量 YOLO 和小型分割模型没有问题
S8 Transformer	可以跑 DeiT-Tiny 这类小模型
S9 大语言模型	0.5B–1.5B 是舒适区，0.5B INT4 优化后预期 40–60 tok/s；7B INT4 约 5 tok/s，属于论文级挑战，4 GB 内存只够裸机跑，所以放到 FZ5D 上做

两块板怎么配合

阶段	在 FZ5D 上做	在 AXU4EVB-E 上对照，体会什么
S1 上板入门	LED 和按键实验，走通综合到 JTAG 下载	第一个实验先在 AXU4EVB-E 上照教程跑通再移植。体会：同一份 RTL 换板只改约束文件，引脚、电平标准、时钟源都要对着原理图重查
S2 FPGA 与 AXI	建工程模板，Zynq PS + 自定义 AXI-Lite IP	同一个工程在两块板上各建一份。体会：PS 配置（DDR 型号和位宽、参考时钟、MIO 外设）是板级的，换板最容易出错的就是它；同一个阵列在 1248 和 728 DSP 上能放多大
S3 HLS	GEMM 上板实测	同一份 HLS 代码换器件综合。体会：pragma 的效果、能达到的 II 和资源占用会随器件变化

阶段	在 FZ5D 上做	在 AXU4EVB-E 上对照，体会什么
S4 加速器设计	按 XCZU5EV 做设计文档	在同一张 Roofline 上画 XCZU4EV。体会：带宽顶相同、算力顶不同，哪些层的瓶颈会因此改变
S5 Linux	用 XSA 自建 2023.1 镜像	AXU4EVB-E 用厂家的现成镜像。体会：厂家 BSP 成熟度对工作量的影响，两块板设备树的差别
S6 分类	DPU 单核 B4096，自研加速器	DPU 放 B1600–B2304。体会：DPU 规模和吞吐、功耗、能效的关系，得到一张随 DSP 缩放的表
S7 检测与分割	自研加速器，USB / HDMI 视频输入	加速器按 728 DSP 重新参数化；条件允许时走 MIPI 摄像头。体会：参数化设计能不能缩放，不同视频输入链路的延迟
S8 Transformer	比 DeiT-Tiny 大一档的模型	可选：同一模型缩小阵列放进去，看资源不够时性能怎么掉
S9 大语言模型	0.5B–1.5B 主线，7B INT4 挑战（8 GB 内存可在 Linux 下跑）	同一个模型测 tok/s，再试“KV Cache 放 PL DDR”。体会：带宽相同所以 tok/s 基本一样，亲手验证解码只看带宽；多一块独占内存能带来什么

如果 FZ5D 一开始就卡住（最可能的原因是找不到米尔的 Vivado 工程或 HDF，PS 侧 DDR 配置推不出来），先用 AXU4EVB-E 继续往下走，同时向米尔要资料，拿到后再换回来。两块板芯片同代、工具相同，换来换去不损失已经做的工作。

两块板都不够用时

板卡	芯片	关键资源	价格	覆盖阶段	说明
ZCU104 / ZCU102	Zynq UltraScale+	1728 / 2520 DSP	万元级	S6–S8，资源更宽裕	Vitis AI 官方支持，论文实验常用。
Alveo U55C / U280	Virtex UltraScale+ HBM	9024 DSP，HBM2 约 460 GB/s，PCIe 插卡	数万元	S9 想把解码速度提高一个数量级时	LLM 解码需要的高带宽。插在服务器上用，不是独立开发板。
Versal VEK280	Versal AI Edge（AIE-ML）	AI Engine 阵列 + 可编程逻辑	数万元	S8–S9	AI Engine 的编程模型和 HLS 不同，需要额外学习成本。

硬件周边技能要不要学

FPGA 和嵌入式领域的技能很多，但你的目标是部署 AI 模型。判断标准只有一个：它会不会出现在“模型 → 加速器 → 板上运行”这条链路上。

要学	**FPGA 的数据通路：DMA、AXI-Stream、跨时钟域、视频输入** AI 部署的瓶颈经常不在计算，而在数据怎么进出 FPGA。这部分是传统 FPGA 工程里和你最相关的，已经放进 S2、S5、S7，不用另开一条线。
按项目再学	**高速数据采集：ADC 接口、LVDS、JESD204、千兆网或 PCIe 采集卡** 只有应用需要从传感器直接采数据再推理时才用得上，比如雷达、工业检测、医疗信号。等项目确定再补，有 S2 的基础大约 2–4 周能上手。纯摄像头图像输入用不到。
按项目再学	**传统数字信号处理：FIR、FFT、DDS 等** 模型输入是雷达、音频、振动这类一维信号时，把预处理放在 FPGA 上很有价值；做图像就跳过。图像预处理（缩放、颜色转换）已经包含在 S7 里。
不用学	**STM32 等单片机** Zynq 的 ARM 端跑 Linux，你需要的是嵌入式 Linux（交叉编译、设备树、驱动），已经加进 S5。单片机裸机开发和单片机上跑 AI（TinyML）是另一条路线，和 FPGA 加速器的知识几乎不复用。
不用学，会看原理图就够	**PCB 画板** 开发板和核心板完全够用，你的 AXU4EVB-E 本身就是核心板（ACU4EV）加载板。DDR、高速串行这类高速板设计是专门的硬件工程，和你的目标无关。你只需要读懂板卡用户手册和原理图：引脚约束、时钟从哪来、外设接在哪，一两天就够。将来真要做定制载板，找硬件工程师合作。
不用补	**模拟电路** 只在画板和设计 ADC 前端时才用得上。数字设计需要的电气常识（时钟、电平、上下拉），S1 和看原理图时顺带就会了。

量化与数据格式约定

量化结果的格式直接决定硬件怎么设计。每一项都先按最常见的默认做法来，自己用公开工具完成量化，不等量化同学； 跑通之后再尝试后面列的方向，同学的成果到了再对照替换。勾选表示这一项已经按默认值落实。

- [ ] **位宽** 默认：权重和激活都用 INT8，累加器 INT32。
  之后可试：W4A8、逐层混合精度
- [ ] **量化方式** 默认：对称量化；权重按输出通道（per-channel），激活按整个张量（per-tensor）。
  之后可试：LLM 权重按 group 量化，group 大小 64 或 128
- [ ] **scale 的表示** 默认：量化工具给出浮点 scale，部署时换算成定点乘数加右移位数，硬件里只做整数运算。
  之后可试：2 的幂 scale，乘法直接变移位
- [ ] **舍入与溢出** 默认：和所用量化工具的舍入方式保持一致；溢出时饱和到 [-128, 127]。
- [ ] **稀疏模式** 默认：第一版不做稀疏；第二版用 2:4 结构化稀疏（每 4 个权重里保留 2 个）。
  之后可试：块稀疏、更高稀疏率的 N:M
- [ ] **存储格式** 默认：稠密权重按 AXI 总线位宽对齐打包；2:4 稀疏每组存 2 个值加 2 个 2 位索引。
  之后可试：bitmask、CSR 等通用稀疏格式
- [ ] **非线性层** 默认：CNN 全部整数化；Transformer 的 softmax、LayerNorm 先用 FP16 或定点，跑通后再整数化。
  之后可试：I-BERT 式全整数近似
- [ ] **导出格式** 默认：ONNX（QDQ 格式）加权重二进制文件，再加一个描述每层 scale 和形状的 JSON。
- [ ] **逐层比对数据** 默认：用 PyTorch 导出每层整数域的输入和输出，作为硬件逐层比对的 golden。
- [ ] **精度验收线** 默认：INT8 分类模型 Top-1 相对浮点下降不超过 1 个百分点。
  之后可试：INT4、稀疏版本单独定线

总体讨论

对整条路线的疑问、想调整的地方写在这里。和各阶段下的笔记一起，自动保存到 `notes/讨论记录.md` 。

各阶段小时数为零基础下的估算投入。路线内容在 `web/data.js` ，改完刷新页面即可；运行 `./make_pdf.sh` 可重新导出 PDF。